Programmierparadigmen

Universität Stuttgart

Sommersemester 2026

Inhaltsverzeichnis

1 Preface **6**

2 Alias list **6**

3 Basics/ Organizational **6**

4 History **6**
4.1 Bits . . . 6
4.2 Assembly . . . 6
4.3 High level languages . . . 6
4.4 Today . . . 6

5 Spectrum of PL **7**
5.1 PL types . . . 7
5.2 Execution types . . . 7

6 Syntax vs. Semantics **7**
6.1 Syntax in Different Programming Languages . . . 7
6.1.1 Four Concepts to Specify Syntax . . . 8

7 Syntax **8**
7.1 Tokens . . . 8
Example (C) . . . 8
7.1.1 Identifiers in Different Languages . . . 8
Naming Conventions . . . 8
7.1.2 Formatting . . . 8
7.2 Context-Free Grammars (CFG) . . . 8
Example: Arithmetic expressions . . . 9
Conclusion . . . 9
7.2.1 Definition . . . 9
7.2.2 Extensions of CFG Syntax . . . 9
7.2.3 Derivations . . . 9
7.2.4 Parse Trees . . . 9
Remark . . . 9
7.3 Scanning . . . 9
7.3.1 Implementing a Scanner . . . 10
7.3.2 Option 1: Ad-hoc Scanner . . . 10
7.3.3 Option 2: Finite Automata . . . 10
From DFA to Scanner . . . 10
Multiple Tokens . . . 10
Longest Match Rule . . . 10

7.4 Parsing . . . 10
7.4.1 Top-down Parsing . . . 10
7.4.2 Bottom-up Parsing . . . 11
7.4.3 Parser Classification . . . 11
7.4.4 LL(k) Parsers (Top-down) . . . 11
Approaches . . . 11
Recursive Descent Parser . . . 11
7.4.5 Example: Recursive Descent Parser . . . 12
Grammar . . . 12
7.4.6 Generating a Top-Down Parser . . . 12
FIRST(N) . . . 12
FOLLOW(N) . . . 13
Computing FOLLOW sets . . . 13
PREDICT sets . . . 13
Computing the Parse Table (LL(1)) . . . 14
7.4.7 Bottom-up Parser . . . 14
Shift Reduce Algorithm . . . 14
Table-based LR parsing . . . 14

8 Names, Scopes, Bindings 15
8.1 Bindings . . . 15
8.2 Scopes . . . 15
8.3 Object Lifetime . . . 15
Object Lifetime vs Binding Lifetime . . . 15
8.4 Storage Allocation . . . 15
8.4.1 Statically Allocated Memory . . . 15
8.4.2 Stack Memory . . . 16
8.4.3 Heap-Based Allocation . . . 16
8.5 Scoping Rules . . . 16
8.5.1 Nested Scope . . . 16
8.5.2 Static vs Dynamic Scoping . . . 16
Static Scoping (Lexical Scoping) . . . 16
Dynamic Scoping . . . 16
8.5.3 Built-in Objects . . . 16
8.6 Aliasing and Overloading . . . 17
8.6.1 Aliasing . . . 17
8.6.2 Overloading . . . 17
8.7 Referencing Environment . . . 17
8.7.1 Shallow Binding . . . 17
8.7.2 Deep Binding . . . 17
Closure . . . 18

9 Control Flow 18
9.1 Expressions . . . 18
9.1.1 Operator vs Operand . . . 18
9.1.2 Notations . . . 18
9.1.3 Multiplicity . . . 18
9.1.4 Order of Evaluation . . . 19
9.1.5 Ordering within expressions . . . 19
9.2 Short circuit evaluation . . . 19
9.3 Control flow with goto statements . . . 19
9.3.1 Continuations . . . 20
9.4 Selection . . . 20
9.5 Iteration . . . 20

9.5.1 Iterators . . . 20
9.6 Recursion . . . 21
9.6.1 Tail Recursion . . . 21

10 Type Systems **21**
10.1 Why Do We Need Types? . . . 21
10.2 Type Checking Strategies . . . 21
10.3 Polymorphism . . . 22
10.4 Special Types and Values . . . 22
10.5 Type Equivalence . . . 22
10.5.1 Structural Equivalence . . . 22
10.5.2 Name Equivalence . . . 22
10.6 Type Compatibility and Conversions . . . 22
10.6.1 Type Compatibility . . . 22
10.6.2 Type Conversions . . . 23
Runetime behavior of conversions . . . 23
10.6.3 Coercions in C . . . 23
10.6.4 Coercions in Javascript . . . 23

11 Formally Defined Type Systems **23**
11.1 Typed Expressions: Syntax . . . 23
Type Checking Expressions: . . . 24
11.2 The Lambda Calculus (λ-Calculus) . . . 24
11.2.1 Functional Abstraction . . . 24
Notation: . . . 24
11.2.2 Grammar (Untyped) . . . 25
Structural Precedence Rules: . . . 25
Variable Scope: . . . 25
11.2.3 Semantics & Evaluation . . . 25
11.2.4 Church Encodings . . . 25
Church Numerals: . . . 25
Church Booleans: . . . 26
Conditional Branching (If-Else): . . . 26
11.2.5 Types in Lambda Calculus . . . 26
Syntax of Types: . . . 26
Type Annotations: . . . 26
Example: . . . 26
11.2.6 Typing Contexts . . . 26
11.2.7 Typing rules . . . 27

12 Composite Types **27**
12.1 Records . . . 27
12.1.1 Variants . . . 28
12.1.2 Memory Layout . . . 28
Packing and Recording . . . 28
12.1.3 Nested Records . . . 29
12.1.4 Reference Model vs Value Model . . . 29
12.1.5 Variant Records (Unions) . . . 29
Use cases . . . 29
12.2 Arrays . . . 29
12.2.1 Operations on arrays . . . 29
12.3 Pointers and recursive types . . . 30
12.3.1 Operations on Pointers . . . 30
12.3.2 Pointers and arrays in C . . . 30

12.3.3 Dangling References . . . 30
12.3.4 Garbage collection . . . 31
Reference Counts: . . . 31
Circular dependencies: . . . 31
Mark and sweep: . . . 31
Optimizations and other algorithms . . . 31

13 Control Abstraction **31**
13.1 Control vs Data Abstraction . . . 31
13.2 Terminology . . . 31
13.3 Reminder: Stack Layout . . . 32
13.4 Call Sequence tasks . . . 32
13.5 Typical Calling Sequence . . . 32
13.6 Stack smashing . . . 33
13.7 Coroutines . . . 33
13.7.1 Coroutines vs Threads . . . 33
13.7.2 Coroutines vs Continuations . . . 33
13.8 Asynchrony . . . 33
13.8.1 Motivation . . . 33
13.8.2 Expressing Asynchrony . . . 33

14 Data Abstraction **33**
14.1 Classes and Objects . . . 34
14.1.1 Inheritance . . . 34
Subclasses vs Subtypes: . . . 34
Liskov's Substitutability Principle . . . 34
Modifying inherited Members: . . . 34
14.1.2 Initialization . . . 34
Superclass constructors: . . . 34
14.1.3 Destructors . . . 35
Execution order of destructors . . . 35
14.1.4 Finalization . . . 35
14.1.5 Dynamic Method Binding . . . 35
Static vs Dynamic . . . 35
Static method binding . . . 35
Dynamic method binding . . . 35

15 Concurrency **35**
15.1 Motivation . . . 35
15.2 Levels of Parellelism . . . 35
15.3 Data Race . . . 36
15.4 Processes, Threads, Tasks . . . 36
15.5 Communication . . . 36
15.5.1 Thread Creation and Management . . . 36
Thread Pools . . . 37
Spawn and Sync in Cilk . . . 37
15.6 Implementing Synchronization . . . 37
15.6.1 Spinning vs Blocking . . . 37
15.6.2 Goals . . . 37
15.6.3 Synchronization vs Parallelism . . . 38
15.6.4 Test-and-set . . . 38
15.6.5 Synchronization Constructs in PLSs . . . 38
Sync in Parallel Loops . . . 38
15.6.6 Implicit Synchronization . . . 38

15.6.7 Memory Consistency . . . 38
Relaxed Memory Models . . . 38
Memory Models of different PLs . . . 38
15.7 Monitors . . . 39
15.7.1 Monitors in Java . . . 39
15.8 Conditional Critical Regions . . . 39
15.9 Transactional Memory . . . 39

16 Functional languages **39**
16.1 Functional Paradigm . . . 39
16.2 Historic origins . . . 39
16.3 Features . . . 40
16.4 Purely Functional PLs . . . 40
16.5 Non-Pure functional PLs . . . 40
16.6 Function Application . . . 40
16.7 Creating Functions . . . 40
16.8 Bindings . . . 40
16.9 Conditional expression . . . 41
16.10 Dynamic Typing . . . 41
16.11 Lists . . . 41
16.12 Assignments . . . 41
16.13 Sequencing . . . 42
16.14 Iteration . . . 42
16.15 Programs as Lists . . . 42
16.16 Evaluation order . . . 42
16.16.1 Lazy Evaluation . . . 42

1 Preface

The idea of these notes is to be written in english, but it is possible that some words appear in German/Spanish, given that english is not my frst language.

These notes contain all topics I valued as important for the exam, but some stuff that may have been trivial for me may not appear in the notes. So if anybody is using my notes to prepare for the exam, please be wary of this.

2 Alias list

- PL: Programming Language

3 Basics/ Organizational

This lecture is about concepts for each language, not a deep dive on a specific language, but an understanding of tradeoffs and features

- Final exam in english (can be answered in german too), but lectures in german
- 6 excersises, 14 days to solve
- Quizzes during each lecture (give bonus points for schein)
- To go to the exam there are 6 excersises 100 points each
  - 30 points in 5 excersises
  - 360 total points
- All help allowed for the exam except electronic. Not sure if printed stuff is allowed.

4 History

4.1 Bits

First computers were programmed in machine language, as a sequence of bits. Machine time was more valuable than developer time.

4.2 Assembly

Human-readable abbreviations for machine language instructions. Each new machine had different assembly language but it was less error prone than raw bits.

The developer is forced o think in terms of very low level operations, directly interacting with the hardware

4.3 High level languages

First high level languages were Fortran, Lisp... More logical and mathematical abstractions.

4.4 Today

1000s of languages, each with their own toolkit and feature set

5 Spectrum of PL

5.1 PL types

- Declarative: Haskell, SQL
- Impertative: Java, C
- Logical (more mathematical): Prolog

5.2 Execution types

- Interpretation (basic Lisp interpreter)
- Compilation (C)
- Mix of both (Java has VM with just-in-time compilation, Python is compiled to bytecode then interpreted)

6 Syntax vs. Semantics

This describes the difference between the structure of code and its meaning.

- **Syntax**: how code is written (structure, rules)
- **Semantics**: what the code actually means / does

6.1 Syntax in Different Programming Languages

Most programming languages have different syntax but similar semantics.
Example:
Java:

```
if (foo > 100) {
  System.out.println("Hello World!");
}
```

Bash:

```
if [ $foo -gt 100 ]
then
  echo "Hello World!"
fi
```

However, sometimes languages share similar syntax but differ in semantics.
Example: Java vs. JavaScript

```
if ("abc" != 5) {
  ...
}
```

- In Java: type error at compile time
- In JavaScript: implicit type conversion is performed (no error)

6.1.1 Four Concepts to Specify Syntax

- Concatenation
- Alternation / choice
- Repetition (Kleene closure)
- Recursion

These concepts form the basis of regular expressions and extend to context-free grammars.

7 Syntax

7.1 Tokens

Tokens are the basic building blocks of a programming language.

Example (C)

- Keywords: `double`, `if`, `return`, `struct`
- Punctuators: `,`
- Literals: `'x'`
- Identifiers: `printf`

Tokens are typically specified using regular expressions (see Theo).

7.1.1 Identifiers in Different Languages

- Some languages (e.g. Ada, Pascal) are case-insensitive
- Letters and digits are almost always allowed
- Underscores are usually allowed

Naming Conventions

- Java:
  - Classes: `ClassName`
  - Variables: `variableName`

7.1.2 Formatting

Depending on the language, indentation and line breaks may or may not affect program behavior.

7.2 Context-Free Grammars (CFG)

Question: Are regular expressions sufficient?
Answer: No.

Example: Arithmetic expressions

$$5 + 7$$

$$(5 + 7) + 6$$

$$((5 + 7) + 6) - 23$$

These expressions grow in nested structures ("boxes"), which require recursion. Regular expressions do not support recursion.

Conclusion CFG = regular expressions + recursion

7.2.1 Definition

$$G = (N, T, R, s)$$

- N: non-terminals (finite set)
- T: terminals (finite set)
- R: production rules, $N \to (N \cup T)^*$
- s: start symbol

(Details covered more deeply in Theo 1.)

7.2.2 Extensions of CFG Syntax

- Kleene star ($*$)
- Kleene plus ($+$)
- Alternation ($|$)

7.2.3 Derivations

A derivation is the process of generating a string from a grammar by repeatedly applying production rules until only terminals remain.

7.2.4 Parse Trees

A tree representation of a derivation:

- Root: start symbol
- Inner nodes: non-terminals
- Leaves: terminals (tokens)

Remark A language can have infinitely many grammars, some of which are ambiguous (multiple derivations for the same string). A good grammar reflects the internal structure of the programming language.

7.3 Scanning

Converting a sequence of characters in the source code into a sequence of tokens. This is done using regular expressions to specify tokens.

7.3.1 Implementing a Scanner

Read one character at a time. If a token is recognized, return it and continue. Otherwise, raise an error.

7.3.2 Option 1: Ad-hoc Scanner

- Manually implemented
- Handles common tokens first
- Used in many production compilers due to compact code and efficiency

7.3.3 Option 2: Finite Automata

- Each token is specified by a regular expression
- Finite automata = recognizers of regular expressions

Pipeline:

$$\text{RegEx} \rightarrow \text{NFA} \rightarrow \text{DFA} \rightarrow \text{Minimal DFA}$$

From DFA to Scanner

- **Switch-based implementation**

```
loop:
  switch (state) {
    case S1:
      switch (character) {
        case 'a': state = S2; break;
        default: error();
      }
  }
```

- **Table-based scanner**

  Table represents states and transitions; can use a generic driver program.

State	'a'	'b'	Return
S1	S2	-	-

Multiple Tokens So far: only one kind of token is recognized. Goal: recognize multiple token types.

Idea: Merge multiple token-level automata into a single automaton.

Longest Match Rule Example: 3.1 vs. 3.1415 → choose the longest valid token.

This requires lookahead.

7.4 Parsing

Converting the sequence of tokens into a syntax tree. Specified using a CFG.

7.4.1 Top-down Parsing

- Start from root node
- Expand non-terminals until reaching terminals
- If multiple rules apply, predict

7.4.2 Bottom-up Parsing

- Start from input tokens
- Combine tokens into subtrees
- Build tree upwards

7.4.3 Parser Classification

	LL(k)	LR(k)
Parse tree construction	Top-down	Bottom-up
Scanning	Left-to-right	Left-to-right
Derivation	Left-most	Right-most
Algorithm	Predictive	Shift-reduce

7.4.4 LL(k) Parsers (Top-down)

Left-to-right scanning, **L**eft-most derivation, **k**-token lookahead

Approaches

- Recursive descent parser
- Table-driven LL parser

```
Start with start symbol

Repeat until no more input:
  Given current non-terminal and next k tokens:
    choose production rule R

  For each symbol X in R:
    if X is non-terminal:
      expand X
    else if X is terminal:
      if next token matches X:
        consume token
      else:
        error
```

Recursive Descent Parser

- One function per non-terminal
- Chooses production using lookahead
- Calls functions recursively
- match(): consumes token or raises error

7.4.5 Example: Recursive Descent Parser

Grammar

- $S \to aB$
- $S \to bC$
- $B \to bbC$
- $C \to cc$

```
S() {
  if (inputToken == a) {
    match(a); B();
  } else if (inputToken == b) {
    match(b); C();
  } else {
    error();
  }
}

B() {
  if (inputToken == b) {
    match(b); match(b); C();
  } else {
    error();
  }
}

C() {
  if (inputToken == c) {
    match(c); match(c);
  } else {
    error();
  }
}
```

7.4.6 Generating a Top-Down Parser

- We must predict which production rule to apply
- This is done using **PREDICT** sets, based on FIRST and FOLLOW

FIRST(N) Set of terminals that can appear at the beginning of strings derived from N.

- For a terminal a: FIRST(a) = $\{a\}$ (including ε, but only if it's **only** ε within a single transition)

Example:

- $S \to$ simple | begin S end
- FIRST(S) = $\{\text{simple}, \text{begin}\}$

FOLLOW(N) Set of terminals that can appear immediately to the right of N in some sentential form. Only applicable to non-terminals

- Includes the end-of-input symbol \$
- Never contains ε

Example:

- $S \to$ aBc
- $B \to d$
- FOLLOW(S) = {\$}
- FOLLOW(B) = {c}

Computing FOLLOW sets Apply the following rules until no changes occur:

1. If A is the start symbol, add \$ to FOLLOW(A)
2. If $B \to \alpha A \beta$, add FIRST(β) $\setminus \{\varepsilon\}$ to FOLLOW(A)
3. If $B \to \alpha A$, or $B \to \alpha A \beta$ with $\beta \Rightarrow^* \varepsilon$, add FOLLOW($B$) to FOLLOW($A$)

Here is an example

S → a S e | B
B → b B C f | C
C → c C g | d | ε

FIRST(C) = {c,d,ε}
FIRST(B) = {b,c,d,ε}
FIRST(S) = {a,b,c,d,ε}

- FOLLOW(C) = {f,g} ∪ FOLLOW(B) = {c,d,e,f,g,\$}
- FOLLOW(B) = {c,d,f} ∪ FOLLOW(S) = {c,d,e,f,\$}
- FOLLOW(S) = {\$, e }

PREDICT sets Determine when a production should be used in an LL(1) parser.

- If the next input token is in PREDICT($A \to \alpha$), apply the rule $A \to \alpha$

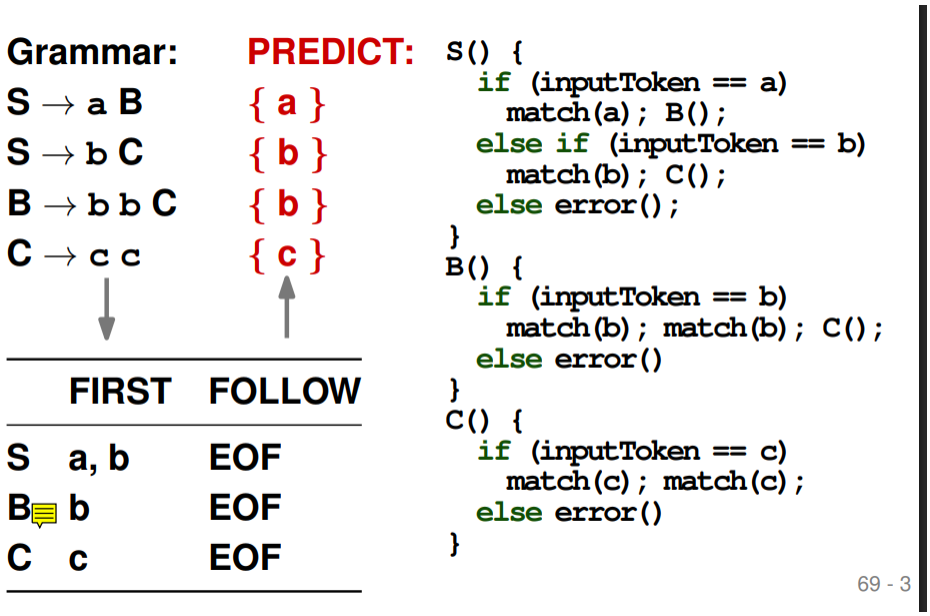
For a production $A \to \alpha$:

- If $\varepsilon \in$ FIRST(α):

$$\text{PREDICT}(A \to \alpha) = \big(\text{FIRST}(\alpha) \setminus \{\varepsilon\}\big) \cup \text{FOLLOW}(A)$$

- Otherwise:

$$\text{PREDICT}(A \to \alpha) = \text{FIRST}(\alpha)$$



Computing the Parse Table (LL(1))

- Define a table $M : N \times T \to$ {production rule, error}
- For each production $A \to \alpha$:
  - For each terminal $t \in$ PREDICT$(A \to \alpha)$:

$$M[A][t] = A \to \alpha$$

And then for the actual parsing, we use like a stack-based approach. And for each iteration we end up looking up a table entry. If no value is found, error, and no more than one value can be found.

7.4.7 Bottom-up Parser

They are $LR(k)$ parsers, with Left-to-right scanning, Right-most derivation, k-tokens look ahead.
Hard to do by hand, usually done by automatically generated table.

Shift Reduce Algorithm Repeat the following until all tokens read and all symbols reduced to start symbol.

```
Shift(read) input tokens
Try to reduce group of symbols into single non-terminal
```

This has sometimes issues, because there can be multiple 'paths' to take.

Table-based LR parsing You have two tables

- Action table: *state* $\times T \to$reduce/shift/accept/error
- Goto table: *state* $\times N \to$ *state*

Then also use a stack of (symbol,state) pairs to record what was seen in the past
These tables are generated using a **characteristic finite state machine**

8 Names, Scopes, Bindings

8.1 Bindings

A binding is an association between a name and an entity. For example, a variable name is bound to a memory location (object).

Different programming languages define different binding rules.

—

8.2 Scopes

- **Scope of a binding:** The textual region where a specific binding is active
- **Scope:** The maximal region where bindings do not change

—

8.3 Object Lifetime

- Global variables: exist for the entire program execution
- Local variables: exist during function execution (or scope lifetime)

Object Lifetime vs Binding Lifetime A single object can be bound multiple times. This means bindings can overlap or coexist, even if the object itself is the same.

—

8.4 Storage Allocation

Three main types of memory:

- **Static:** Fixed (absolute) addresses
- **Stack:** Used for function calls (allocated on call, deallocated on return)
- **Heap:** Dynamically allocated memory at arbitrary times (slower, more flexible)

—

8.4.1 Statically Allocated Memory

Depends on the language, but typically used for:

- Global variables
- Constant literals
- Symbol tables
- etc.

—

8.4.2 Stack Memory

Think of it as a stack of function calls.

Each function call creates a **stack frame**, which contains:

- Function arguments
- Local variables
- Temporary values
- Return address

—

8.4.3 Heap-Based Allocation

Used for objects whose size is not known at compile time.

Example: dynamically allocated data structures.

Some programming languages use **managed memory**:

- Unreachable objects are automatically deallocated (garbage collection)
- Easier to use, but less control and potential overhead

—

8.5 Scoping Rules

Scoping rules determine which bindings are active at a given point.

8.5.1 Nested Scope

Inner (deeper) scopes usually override outer ones.

—

8.5.2 Static vs Dynamic Scoping

Static Scoping (Lexical Scoping) Binding is determined by the program text.

You resolve a name by moving outward through nested scopes.

Important: Order inside a scope usually does not matter (a declaration below can still be used above, depending on the language).

—

Dynamic Scoping Binding depends on the **control flow** at runtime.

You resolve a name by walking up the **call stack** until a binding is found.

—

8.5.3 Built-in Objects

Most programming languages have built-in objects/functions:

- They exist in the outermost scope
- Accessible from everywhere

They can often be overwritten → this is called **Monkey Patching**.

—

8.6 Aliasing and Overloading

8.6.1 Aliasing

Two or more names refer to the same memory location.

```
double n = 5.0;
double* p = &n;
```

—

8.6.2 Overloading

Multiple definitions for the same name (usually functions).

```
class Overload {
  void foo() {}
  void foo(int n) {}
  void foo(String s) {}
}
```

Which function is called depends on the arguments.

—

8.7 Referencing Environment

The complete set of bindings visible at a specific point in a program.

This is determined by the scoping rules.

```
function a() {
  var x = 23;
  function b() {
    console.log(x);
  }
  x = 42;
  return b;
}

b = a();
var x = 5;
b();
```

—

8.7.1 Shallow Binding

Referencing environment is created when the function is **called**. Used with dynamic scoping.

—

8.7.2 Deep Binding

Referencing environment is created when the function reference is **created**. Used with static scoping.

Closure : Its a structure to implement deep binding. **Pointer to function code + Pointer to referencing environment** —

9 Control Flow

Control flow defines the order in which instructions are executed:

- Sequencing
- Iteration
- Recursion
- Concurrency
- Exceptions

—

9.1 Expressions

9.1.1 Operator vs Operand

- **Operator:** Symbol/function that performs an operation (e.g. +, ∗)
- **Operand:** The values the operator acts on

—

9.1.2 Notations

- **Prefix**
  - `op a b`, `op(a,b)`, `(op a b)`
- **Infix**
  - `a op b`
- **Postfix**
  - `a b op`

—

9.1.3 Multiplicity

Number of operands an operator expects:

- Unary: `a++`
- Binary: `a + b`
- Ternary: `condition ? a : b`

—

9.1.4 Order of Evaluation

Applies to arithmetic, boolean, and pointer expressions.
Defined by:

- **Precedence:** Which operators bind stronger
- **Associativity:** Direction of evaluation for same-precedence operators

  Typical rules:

  – Arithmetic operators → usually left-to-right (except exponentiation)
  – Assignment → usually right-to-left

Operator	Meaning
++, −−	Post-increment, post-decrement
++, −−	Pre-increment, pre-decrement
*	Pointer dereference
<, >	Inequality test
==, !=	Equality test
&&	Logical and
||	Logical or
=, +=	Assignment



9.1.5 Ordering within expressions

$$a - f(b) - c * d$$

Imagine f alters one of the other variables. What is the orden there?
Solution: Depends on language

- Java, C#: Left-to-right
- C: undefined, compiler may pick the best order

9.2 Short circuit evaluation

Saves time when evaluating booleans.
The usual we already know $A\&\&B$, if A is false B is not computed.
Almost all PL's implement this (except notably Pascal)

9.3 Control flow with goto statements

Most assembly is done with conditional/unconditional jumps
Early PL's implemented goto. It jumps to statament label.
According to Djikstra, you should avoid them and use structured control flows.

9.3.1 Continuations

Generalization of goto's

It allows to define new control flow structures

- Exceptions
- Iterators
- Coroutines

High-level definition: Context in which to continue executions
Low-level definition (three parts)

- Code adress: where to continue
- Referencing environment
- Another Continuation (to be used when code returns)

Really complicated to grasp and work with, they are heavily discouraged.

9.4 Selection

Branch that depends on a condition (if/else, switch/case)

These are mapped to their correspondent goto statements in the compiler.

9.5 Iteration

There are two forms of loops

- Enumeration-controlled: Once per value in finite set. Usually built with initial value, bound, step size.
- Logically-controlled: Repeat until boolean expression is false. The condition can be placed pre-test, mid-test or post-test

And then there are PL specific traits, like modification of variables within loop, leaving loop, skipping loop, etc.

9.5.1 Iterators

Special enumeration-controlled loop. Iterates through any kind of set/sequence of values. Allows to enumerate and use the individual values

Divides itself in:

- True Iterators: Subrutine with *yield* statements (each yield returns another element). Used in python for loops
- Iterator Objects: Regular Objects with methods for initialization, next value getter, and test for completion. Used in Java and C++
- First Class Functions: Divides into two functions:
  - One function says what to do for each element
  - Another function calls the first function for each element

  Used in java streams for example

9.6 Recursion

Powerful construct equally as powerful as iteration. Usually less efficient than iteration, because of the stack frame overhead.

9.6.1 Tail Recursion

When the recursive call is the last statement before the return. The compiled code can reuse the stack frame.

10 Type Systems

In most programming languages, expressions and memory objects are associated with **types**. For example, the assignment $x = 4$ typically implies that x belongs to a numeric type.

10.1 Why Do We Need Types?

Types serve several critical functions in software development:

1. **Operational Context:** They define how operations are interpreted. For instance, $a + b$ might perform mathematical addition for integers but string concatenation for text.
2. **Safety Constraints:** They prevent invalid operations, such as attempting to calculate the logarithm of a set.
3. **Readability:** They act as documentation, making code easier to understand (though they can occasionally make code more verbose to write).
4. **Performance:** They allow the compiler to perform specific machine-level optimizations.

   Note on Raw Data: At the lowest level, bits and bytes are **untyped**. Consequently, assembly languages are generally untyped; the 'type' is only imposed by how the instructions treat the data.

10.2 Type Checking Strategies

Type checking ensures a program adheres to its language's compatibility rules. A violation results in a **Type Error**.

```
int a = 3;
String b = a - 2; // Results in a Type Error
```

- **Strongly Typed:** Prevents operations on improper types. Most modern languages follow this, though C allows some exceptions (like variable-parameter functions).
- **Statically Typed:** Types are checked primarily at **compile-time**. While no language is 100% static, most prioritize early error detection.
- **Dynamically Typed:** Type checking is deferred to the **runtime**. This is common in scripting languages like Python or JavaScript.
- **Gradual Typing:** A middle ground where type annotations are optional, allowing tools to check for obvious errors while maintaining flexibility.

10.3 Polymorphism

There are two primary ways a language handles different types through a single interface:

- **Parametric Polymorphism:** Code is written generally to take a set of types as a parameter (e.g., Generics in Java: `Set<String> a;`).
- **Subtype Polymorphism:** A type is defined as an extension or refinement of a supertype (e.g., class inheritance).

10.4 Special Types and Values

- **void:** Represents the absence of a type; typically has only one possible value.
- **null:** Indicates that a variable does not currently hold a valid value of its assigned type.
- **Option:** A container indicating that a value may or may not be present, helping to avoid null-related errors.

10.5 Type Equivalence

This determines if two types are considered 'the same.' Strategies vary across languages based on naming or memory representation.

10.5.1 Structural Equivalence

Types are equivalent if they share the same internal structure. **Issue:** It cannot distinguish between different concepts that look identical.

```
type student = record { name: string; age: integer }
type school  = record { name: string; age: integer }
// Under structural equivalence, these are the same type.
```

10.5.2 Name Equivalence

Types are only equivalent if they share the same explicit name. This is used in languages like Java to ensure that distinct entities (like a Student and a School) remain separate even if their data fields are identical.

10.6 Type Compatibility and Conversions

10.6.1 Type Compatibility

Check whether combining two values is valid, regarding their types. By combining we mean

1. Assignment
2. Operators
3. Function Calls

Compatible types are not always equal (float vs int).
Each PL defines rules for which types are compatible
Exmaple of rules

- Can assign subtype to supertype
- Different number types are compatible with eachother
- Collections of same type are compatible, even if length differs

10.6.2 Type Conversions

When types aren't equal they must be converted.

- Cast: explicit type conversion. Programmer changes value's type from T1 to T2
- Coercion: implicit type conversion. PL allows value of type T1 in situations where T2 is expected

Runetime behavior of conversions : There are three cases

1. Types are structurally equivalent. No code generated
2. Types have different sets of values but represented the same in memory: May need a check that the value is in target type
3. Different low-level representations: Need special conversion instructions

10.6.3 Coercions in C

Most primitive types are coerced whenever needed. Obviously, some coercions cause information loss ($float \rightarrow int$ loses fraction, $int \rightarrow char$ may cause overflow)

So the implicit conversion order is

$$bool \rightarrow char \rightarrow short\,int \rightarrow unsigned\,int \rightarrow long \rightarrow ...$$

(in x86 at least)

10.6.4 Coercions in Javascript

Because JS is a slop language, basically all types are coerced when needed. This is because the language sucks

11 Formally Defined Type Systems

Formally defined type systems provide a mathematical framework to reason about program behavior, implemented directly within a compiler to guarantee type safety before runtime.

11.1 Typed Expressions: Syntax

We begin by defining a simple language of booleans and numbers via the following structural grammar:

$$t ::= \text{true} \mid \text{false} \mid \text{if } t \text{ then } t \text{ else } t \mid 0 \mid \text{succ } t \mid \text{pred } t \mid \text{iszero } t$$

While we can intuitively guess the evaluation mechanics of these expressions, not all syntactically valid terms make semantic sense (e.g., succ false). We use a binary relation $t : T$ to denote that a term t safely evaluates to a specific type T.

Our core language consists of two base types:

1. Nat: The type representing natural numbers.
2. $Bool$: The type representing boolean logical values.

Type checking is governed by static inference rules. These are visually represented as fractions, where the numerator explicitly defines the **premises** (assumptions) and the denominator defines the **conclusion** (implication):

$$\frac{}{\texttt{true} : Bool}\textbf{(T-True)} \qquad \frac{}{\texttt{0} : Nat}\textbf{(T-Zero)}$$

$$\frac{}{\texttt{false} : Bool}\textbf{(T-False)} \qquad \frac{t_1 : Nat}{\texttt{succ } t_1 : Nat}\textbf{(T-Succ)}$$

$$\frac{t_1 : Bool \quad t_2 : T \quad t_3 : T}{\texttt{if } t_1 \texttt{ then } t_2 \texttt{ else } t_3 : T}\textbf{(T-If)} \qquad \frac{t_1 : Nat}{\texttt{pred } t_1 : Nat}\textbf{(T-Pred)}$$

$$\frac{t_1 : Nat}{\texttt{iszero } t_1 : Bool}\textbf{(T-IsZero)}$$

Type Checking Expressions:

- **Typing Relation:** The smallest binary relation between terms and types that satisfies all valid instances of the structural inference rules. A term t is defined as **typeable** (or **well-typed**) if there exists some valid type T such that $t : T$.
- **Type Derivation:** A structured tree composed of sequential inference rules proving that $t : T$ holds true.

  Example of a Type Derivation Tree: To formally prove that the compound term iszero (succ 0) is well-typed as a $Bool$, we construct a bottom-up derivation tree using our deduction rules:

$$\frac{\dfrac{\overline{0 : Nat}}{\text{succ } 0 : Nat}}{\text{iszero (succ } 0) : Bool}$$

  Each horizontal bar represents the invocation of a typing rule, proving the target term below it is true based on the axioms above it.

11.2 The Lambda Calculus (λ-Calculus)

The λ-calculus is a foundational computational model designed to capture the structural essence of functional programming languages. It acts both as a minimal programming language and as a rigorous mathematical system.

11.2.1 Functional Abstraction

In pure λ-calculus, functions are first-class citizens: everything is a function. Functions accept other functions as arguments and yield functions as structural outputs.

Notation: $\lambda n.\langle\text{result}\rangle$ represents an anonymous function that binds the input parameter n and evaluates the body expression inside $\langle\text{result}\rangle$.

- **Identity Function:** $\lambda x.x$
- **Negation Function:** $\lambda x.$if x then false else true

11.2.2 Grammar (Untyped)

The core syntax of the untyped λ-calculus is remarkably simple, consisting of only three structural terms:

$$t ::= x \mid \lambda x.t \mid t_1\, t_2$$

Where a term is either a local variable (x), a functional **abstraction** ($\lambda x.t$), or a function **application** ($t_1\, t_2$). Explicit parentheses are introduced purely to remove structural ambiguity.

Structural Precedence Rules:

- **Left-Associativity:** Multiple function applications evaluate implicitly from left to right. Thus, the term $s\, t\, u$ is parsed precisely as $((s\, t)\, u)$.
- **Greedy Abstractions:** The scope body of a lambda abstraction spans as far to the right as physically possible. For example, $\lambda x.\lambda y.x\, y\, x$ evaluates as $\lambda x.(\lambda y.((x\, y)\, x))$.

  Understanding Applications: An application represents executing a function with a concrete argument. For example, running the expression $(\lambda x.\text{succ}\ x)\, 5$ substitutes the value 5 into the variable x, returning succ 5 (which evaluates to 6).

Variable Scope: An occurrence of a variable x is **bound** if it falls within the body of an abstraction component λx. If it is not enclosed by a matching abstraction, it is considered a **free variable**.

Formally, the set of free variables within a term t is resolved recursively:

$$\text{freevars}(t) = \begin{cases} \{x\} & \text{if } t = x \\ \text{freevars}(t_1) \setminus \{x\} & \text{if } t = \lambda x.t_1 \\ \text{freevars}(t_1) \cup \text{freevars}(t_2) & \text{if } t = t_1\, t_2 \end{cases}$$

11.2.3 Semantics & Evaluation

Computation in the λ-calculus progresses step-by-step through a process called **β-reduction** (Beta-reduction). When an abstraction is applied to an argument, the parameter replaces all instances of the bound variable inside the body expression:

$$(\lambda x.t_1)\, t_2 \to t_1[x \mapsto t_2] \quad \text{(Read: Replace all free occurrences of } x \text{ in } t_1 \text{ with } t_2\text{)}$$

Standard deterministic computation dictates reducing the outermost, leftmost reducible expression (*redex*) first.

11.2.4 Church Encodings

Church encodings allow us to represent standard primitives and control structures purely through function compositions.

Church Numerals: Natural numbers are encoded by how many times a target function f is iteratively applied to a base argument x:

$$\begin{aligned} 0 &\equiv \lambda f.\lambda x.x \\ 1 &\equiv \lambda f.\lambda x.f\, x \\ 2 &\equiv \lambda f.\lambda x.f\, (f\, x) \end{aligned}$$

Church Booleans: Booleans act as pure structural selection switches that choose between two positional parameters:

$$\begin{aligned} \text{true} &\equiv \lambda a.\lambda b.a \quad \text{(Selects the first argument)} \\ \text{false} &\equiv \lambda a.\lambda b.b \quad \text{(Selects the second argument)} \end{aligned}$$

Conditional Branching (If-Else): Using Church Booleans, conditional logic is written via a dedicated combinator expression:

$$\text{if-then-else} \equiv \lambda a.\lambda b.\lambda c.a\, b\, c$$

> **Why this works:** The conditional parameter a receives two inputs (b and c). If a evaluates to true ($\lambda a.\lambda b.a$), it naturally yields the first option b. If a evaluates to false ($\lambda a.\lambda b.b$), it discards b and outputs the second option c.

11.2.5 Types in Lambda Calculus

Untyped syntax allows for meaningless expressions, such as attempting to execute a boolean variable as a function: true false. Because true is not a function abstraction, this causes an evaluation crash. Conversely, $(\lambda x.x)$ true is mathematically clean.

Syntax of Types: To intercept these structural errors before execution, we formalize a set of basic types:

$$T ::= \mathit{Bool} \mid T_1 \to T_2$$

The function type constructor $\to$ is implicitly right-associative: $T_1 \to T_2 \to T_3$ parses as $T_1 \to (T_2 \to T_3)$.

Type Annotations: To evaluate types statically, bound variables are explicitly annotated inside function abstractions:

$$\lambda x : T_1.t : T_1 \to T_2$$

This states that the function accepts an input argument of type T_1 and evaluates to an internal body of type T_2.

Example:

$$\lambda x : \mathit{Bool}.\ \text{if } x \text{ then true else false} : \mathit{Bool} \to \mathit{Bool}$$

11.2.6 Typing Contexts

To properly determine types for open expressions containing free variables, we extend our simple binary typing relation $(t : T)$ into a ternary relation:

$$\Gamma \vdash t : T$$

This reads: *'Under the typing assumptions stored in the context environment* Γ*, the term* t *possesses type* T*.'*

The **typing context** Γ tracks variable-to-type map assignments for free variables. If an expression contains zero free variables (a closed term), the context is empty, which we write as:

$$\vdash t : T$$

11.2.7 Typing rules

$$\frac{x : T \in \Gamma}{\Gamma \vdash x : T}\textbf{(T-Var)} \qquad \frac{}{\Gamma \vdash \mathbf{0} : Nat}\textbf{(T-Zero)}$$

$$\frac{\Gamma \vdash t_1 : Nat}{\Gamma \vdash \mathbf{succ}\ t_1 : Nat}\textbf{(T-Succ)}$$

$$\frac{\Gamma \cup \{x : T_1\} \vdash t_2 : T_2}{\Gamma \vdash \lambda x : T_1.t_2 : T_1 \to T_2}\textbf{(T-Abs)}$$

$$\frac{\Gamma \vdash t_1 : Nat}{\Gamma \vdash \mathbf{pred}\ t_1 : Nat}\textbf{(T-Pred)}$$

$$\frac{\Gamma \vdash t_1 : T_1 \to T_2 \quad \Gamma \vdash t_2 : T_1}{\Gamma \vdash t_1 t_2 : T_2}\textbf{(T-App)} \qquad \frac{\Gamma \vdash t_1 : Nat}{\Gamma \vdash \mathbf{iszero}\ t_1 : Bool}\textbf{(T-IsZero)}$$

$$\frac{}{\Gamma \vdash \mathbf{true} : Bool}\textbf{(T-True)}$$

$$\frac{}{\Gamma \vdash \mathbf{false} : Bool}\textbf{(T-False)} \qquad \frac{\Gamma \vdash t_1 : Bool \quad \Gamma \vdash t_2 : T \quad \Gamma \vdash t_3 : T}{\Gamma \vdash \text{if } t_1 \text{ then } t_2 \text{ else } t_3 : T}\text{(T-If)}$$

12 Composite Types

Thay are new types that are formed by joining together simpler types and using a **type constructor**

Common type constructors:

- Records
- Arrays
- Strings
- Sets
- Pointers
- Lists

12.1 Records

Also called structures or **structs**

They store and manipulated related data of heterogeneous types together. Each data component is a field. First introduced in Cobol and Algol 68

Example of a struct in C

```
struct element{ // defines record
  char name[2]; // four fields
  int atomic_number;
  double atomic_weight;
  _Bool metallic;
}

struct element copper; // variable of record type
copper.name[0] = 'C';
// ...
if (copper.metallic) {
```

```
  // ...
}
```

12.1.1 Variants

Most PL's offer some type of record

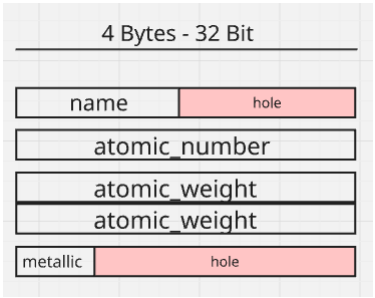
1. C: structs
2. C++: Special form of class
3. Fortran 90: Types
4. OCaml: Tuples (order of fields is irrelevant)
5. Java: Since Java 14 records but with immutable fields

12.1.2 Memory Layout

How are they stored in memory?

- Fields are usually stored in adjacent locations
- Field access is then Adress + Offset
- Alignment Constraints may create 'holes'. E.g 4-byte ints on x86 must start at address divisible by 4

```
struct element{
  char name[2];
  int atomic_number;
  double atomic_weight;
  _Bool metallic;
}
```



In total, 20 bytes for the entire record

Packing and Recording

- Packing: Avoid holes and break alignment. This will require additional instructions to operate on fields
- Reordering: Minimize holes but respect alignment.

12.1.3 Nested Records

Two options

1. Lexically nested

```
struct outer_record{
  char some_field[10];
  struct { // no name for this inner record
    int some_other_field;
    double yet_another_field;
  } nested_field
}
```

2. Fields of record type

```
struct S {
  int i;
  struct T n;
}
```

12.1.4 Reference Model vs Value Model

Occurrence of a variable may mean a reference to its memory location or the value stored in the variable.

Java usually works by reference, C works by value.

12.1.5 Variant Records (Unions)

Special kind of record that reuses the same memory location for multiple variables.

It works under the assumption that variables are never used at the same time, so the size of the record is the size of the largest member.

Use cases : For example, you can use it to store a pixel in different ways (RGB, YCbCr, HEX) and such. Don't need to redundantly use different variables when you can use the same one.

12.2 Arrays

Conceptually it's a mapping from **index type** to **element type**. Takes space in adjacent places in memory.

You can nest them into multiple dimensions.

12.2.1 Operations on arrays

Not going into detail because they are known

1. Slicing
2. Comparison (element-wise) of arrays of equal length
3. Mathematical operations

The memory layout is continuous in memory, and varies a bit for multiple dimensions depending on the PL.

1. C: Row major order. First the first row, then sequentially the second row and so on

2. Fortran: Column Major Order. First the first column, then sequentially the second column and so on.

The layout is essential, as it determines the efficiency of nested loops that iterate through multi-dimensional arrays. Caching can make iterating super efficient, but cache misses make everything significantly slower.

12.3 Pointers and recursive types

- Pointer: Reference to location of memory object (an address)
- Recursive Type: Composite type with reference to objects at the same time

This is strongly linked with the **reference vs value model**

12.3.1 Operations on Pointers

- Creation: We can do it implicitly when calling a constructor, or use a built-in function that allocates heap memory and returns memory to it (malloc in C). We can also use the address-of operator (*).
- Allocating Memory: A pointer itself is only an adress. We need enough memory for our objects. Some PL's make it implicit, others have to be explicitly determined.
- Dereferencing: Access memory object refers to accesing entire object (dereferencing operator). Access fields of record that the pointer refers to (right arrow notation in C++)
- Deallocation: Memory must be reclaimed at some point, otherwise memory leak. Some PL's have explicit deallocation (Rust, C). If it's implicit, it's called **Garbage collection** (Java, C#, Python)

12.3.2 Pointers and arrays in C

They are closely linked language constructs

This is all the same thing (the 4 last lines)

```
int n;
int *a;
int b[5] = {1,2,3,4,5};

a = b;
n = a[3];
n = *(a+3);
n = b[3];
n = *(b+3);
```

Array access is the same thing as pointer arithmetic

$$E1[E2] = (*((E1) + (E2)))$$

So funnily, arr[3] is the same as 3[arr]

The main difference is allocation. Arrays are implicitelly deallocated.

12.3.3 Dangling References

It is a live pointer that no longer points to a valid object. It is dual to memory leaks.

Happens when

1. Pointer to stack object escapes it's surrounding context. (Pointers to local variables)
2. Heap object is explicitely deallocated, but pointer lives on. (using pointer after free)

12.3.4 Garbage collection

It is memory deallocation managed by the PL implementation. It avoids dangling references and such, but costs performance.

Reference Counts: We use one counter for each memory object. Increment when pointer created and decrement when pointer destroyed. Deallocate when count is zero.

Circular dependencies: Something like a doubly linked list have circular references, so even though the object is useless the PL may not recognize that.

So the better approach is saying that object o is useless unless there is a chain of valid pointer

Mark and sweep: Algorithm to identify useless blocks

1. Walk heap and mark every block as useless
2. Start from external references (i.e names in programm) and mark every reachable block as useful
3. Move all useless block to free list.

Optimizations and other algorithms : There are various improvements to mark and sweep:

1. **Pointer reversal:** Eliminates the need for an auxiliary traversal stack during the marking phase. It temporarily reverses the heap pointers themselves to track the return path, achieving $O(1)$ space complexity.
2. **Stop-and-copy:** Divides the heap into two halves (*From-Space* and *To-Space*). When one fills up, execution stops, and livepoly objects are copied compactly into the other half. This completely eliminates memory fragmentation but halves the available heap space.
3. **Generational garbage collection:** Based on the hypothesis that 'most objects die young.' The heap is split into generations (e.g., young and old). The young generation is collected frequently and quickly; survivors are promoted to the old generation, which is scanned less often.

13 Control Abstraction

13.1 Control vs Data Abstraction

A control abstraction abstracts a well defined operation (e.g a subroutine). A Data Abstraction abstracts how to represent information, such as types and classes.

13.2 Terminology

1. Subroutine: Mechanism for control abstraction
   - Function: Subroutine with return value
   - Procedure: Subroutine without return value
2. Parameters
   - Actual parameters: data passed by caller
   - Formal parameters: data recieved by callee
3. Calling Sequences: It's low level code to maintain call stack. Happens four times

(a) Before subroutine call inside the caller
(b) At beginning of subroutine in callee **prologue**
(c) At the end of subroutine in callee **epilogue**
(d) After subroutine inside caller

They are important to understand performance / security implications

13.3 Reminder: Stack Layout

Each procedure call has one stack frame made up of

1. Frame pointer: The base address used to access data in current stack frame
2. Stack Pointer: First unused location in current stack frame

13.4 Call Sequence tasks

It's tasks are the following

1. Pass parameters and return values
2. Update program counter
3. Save return address
4. Save and restore registers
5. Updated stack and frame pointers

13.5 Typical Calling Sequence

1. Tasks done by caller before call
    - Save registers whose values may be needed later
    - Compute values of arguments and move them into stack/registers
    - Pass return address and jump to subroutine
2. Tasks in prologue
    - Allocate a frame
    - Save old frame pointer on stack and update to point to the new frame
    - Save registers from old subroutine to avoid overwriting
3. Steps in epilogue
    - Move return value into register/stack
    - Restore registers
    - Restore frame and stack pointer
    - Jump back to return address
4. Steps done by caller after call
    - Move return value to where it is needed
    - Restore registers

13.6 Stack smashing

It's a special kind of **buffer overflow vulnerability**. If out of bounds checks are wrong, may write beyond space allocated for local variable, allowing to rewrite e.g the return address and jump into malicious code

13.7 Coroutines

It's a control abstraction that allows suspending executing and resuming after suspension. Used to implement multi-tasking.

13.7.1 Coroutines vs Threads

Coroutines have explicit transfer of control, and only one coroutine runs at a time.

In threads, control flow transfered implicitely and preemptively. Multiple threads may run concurrently.

13.7.2 Coroutines vs Continuations

In coroutines, old program counter is saved when suspending, and when resuming we continue where we left off.

In continuations, whenever jump we always start in the same position, so old programm counter is lost.

13.8 Asynchrony

13.8.1 Motivation

Parts of a programm may take very long, so we would want to continue with the rest of the programm in the meantime.

13.8.2 Expressing Asynchrony

- Event-driven programming: Paradigm where the execution of programm is based by events, so ideal for async. The issue is mainly a deeply nested control flow, and error handling is scattered throughout the code. Sync is also tough.
- Promises (futures): It is an object to represent a not yet computed value. Can have one of three states
  - Pending
  - Resolved
  - Rejected

  Control flow easier to understand than event-driven programming. Explicit sync with Promise.all, and all error handling in one place. However, verboser syntax.
- Async and await: Same as futures but with some sugar to make programming with it easier. We label a function async as it performs async computation. It returns a promise, and may await other async functions. Easy error handling

14 Data Abstraction

The idea is to describe a class of memory objects and their associated behavior.

An abstract data type has a set of values and set of operations (e.g **Stack**, with the values being the data on the stack and the operations being push, pop, peek, etc.)

14.1 Classes and Objects

Classes are the most basic way of data abstraction, with encapsulation and information hiding. Objects are:

- Instances of classes in class based PL's (Java)
- Primary entities in prototype-based PL's (JavaScript)

14.1.1 Inheritance

We reuse code by defining a new abstraction as extension of an existing abstraction. A subclass inherits members of superclass, and can add/modify it's members.

Subclasses vs Subtypes: Are subclasses a subtype of the superclass?
In principle, **no**.

- Subclassing is reusing code inside a class
- Subtyping enables code reuse in clients of a class.

But in practice, most PLs merge these concepts.

Liskov's Substitutability Principle : Each subtype should behave like the supertype when being used through the supertype.

- Let B be subtype of A
- Any object of type A could be replaced with type B
- Any programming against A should also work with obejcts of type B

Modifying inherited Members: Depends on the PL, Java allows overriding everything, C++ the elder function must be declared as **virtual**

Can a subclass hide inherited members? In Java, subclass can't **change** the visibility of members. In Eiffel, subclass can both restrict and increase visibility.

In C++

- Subclass can decrease visibility, but not increase it.
- Subclass can hide superclass methods by deleting them.
- Public/protected/private inheritance makes it so that the inherited members are **at most** public/protected/private.

14.1.2 Initialization

A class can have zero, one or more **constructors**. They vary by argument number and type (In Eiffel constructors can vary by name).

In some PL's (Java) constructor must be called explicitly always. In C++ it can sometimes be called implicitly (usually calls zero argument constructor).

Superclass constructors: During initialization of subclass, they also initialize inherited superclass fields. Constructor of superclass is called first.

14.1.3 Destructors

In some PL's each class can define a destructor. It is called when the object is out of scope or the delete operand is called.

Highly recommended if the class dynamically allocates memory (where you must free the memory otherwise memory leak).

Execution order of destructors : The destructor of the subclass is called before the one of the superclass.

14.1.4 Finalization

Java and C# don't have destructors but **finalizers**. They are called inmediatly before object gets garbage-collected. Used to clean up resources like file handles.

In some short running programms it may never even be called.

14.1.5 Dynamic Method Binding

Static vs Dynamic : If we have a subclass that defines a method already defined in the superclass. Which method gets called? (based on type of variable and type of object the variable refers to)

Static method binding : Bind method based on type of variable (can be done at compile time).

Dynamic method binding : Bind method based on the type of object the variable refers to. Can usually only be done at runtime. The correct method to call is found via a **Virtual method table (vtable)**. Every object points to a table with its methods

Each of them has their own pros and cons. PL's like python have dynamic binding for all methods, Java default is dynamic but you can set methods as non-overridable. C++ has static as default, but option for dynamic

In Java, 'final' keyword to dissalow overriding, and in C++, 'virtual' to allow overriding (and use dynamic binding)

15 Concurrency

15.1 Motivation

Mixture of wanting to exploit hardware parallelism, and capturing the logical structure of a problem.

Terminology

- Concurrent: Two or more running tasks whose execution may be at some unpredictable point
- Parallel: Two or more tasks are actively executing at the same time
- Distributed: Physically separated processors

15.2 Levels of Parellelism

- Signals propagating through circuits
- Instruction-level parallelism (e.g loading from memory while another program executes)
- Vector parallelism: GPUs executing single instruction on a vector of data
- Thread-level parallelism

15.3 Data Race

Two accesses of the same shared memory, with at least one of them being a write, and the ordering of the accesses is non-deterministic

15.4 Processes, Threads, Tasks

- Process: Operating system construct that can execute threads
- Thread: Active entity that runs concurrently with other threads (from the programmers view at least)
- Task: Unit of work that must be done by some thread

15.5 Communication

To pass information between threads

- Shared memory
- Message passing

15.5.1 Thread Creation and Management

Depends on the PL, each provides different constructs

- Co-begin: Compound statement where all statements are executed concurrently
- Parallel loops: Loop whose iterations execute concurrently

  To data share here, some PL's can specify which variables are shared among the threads.
- Fork (with join): Fork is the explicit creation of a thread, and join is to wait a previously forked thread to terminate
- POSIX threads aka pthreads: It's a low-level API to create and manage threads. Very flexible, but hard to use. Supported by C and C++.

  Creation

  ```
  #include <pthread.h>

  pthread_t thread_id;
  pthread_create(&thread_id, NULL, thread_function, arg);
  ```

  It has an ID, some optional attibutes, the function to execute in thread, and the arguments to pass in function

  Termination

  ```
  pthread_exit((void *) return_value);
  ```

  It is implicitly called when threads function terminates. You can also block and wait for another thread to join

  ```
  pthread_join(thread_id, NULL);
  ```

- Message passing: It avoids the usual pitfalls of shared memory. Done with two key constructs
  - Independent threads: 'actors' in Java
  - Messages exchanged between threads: 'Channels' in Go and Rust

Thread Pools : Used to separate tasks to execute from how they are executed in threads.

So the thread pool is a set of (idle) threads that can execute tasks. Reduces cost of startup and creation. Lets the pool implementation decide the scheduling of tasks

Spawn and Sync in Cilk : It's an extension of C, where programmer expresses tasks and their dependencies.

- spawn calls function to be executed as logically concurrent task
- sync joins all tasks spawned by ceiling task

The scheduler assigns tasks to processor cores through **work stealing**

15.6 Implementing Synchronization

We need mechanisms to control the relative order of operations on different threads. They must be explicit in shared-memory model (so that the variable being read reads the newest value stored), and implicit in the message-passing model(the sender recieves the message after it has been sent)

15.6.1 Spinning vs Blocking

They are two different forms of Sync

- Spinning (busy-waiting): A thread re-evaluates a condition until it becomes true (because of some other thread)
  - Spin locks: They provide mutual exclusion. In principle they can be implemented with only load and store operations, but super-linear time and space complexity. In practice, it is implemented using special hardware instructions. Read, modify and write a memory location as atomic.

    Naive:

    ```
    // Lock acquisition
    while (true):
      if (L == 0):
        L = 1
        break
    //Lock release
    L = 0
    ```

    The issue is that multiple threads may enter the if- at the same time
  - Barriers: No threads continue until all threads have reached specific point. It's based on atomic fetch-and-decrement.

    Initialize the shared countert to n. Decrement when thread reaches barrier. Last to arrive flips shared boolean that deactivates the barrier
- Blocking: Waiting threads stop computation until some condition becomes true, then the scheduler reactivates the thread

15.6.2 Goals

- Make some operations atomic, either all hapens or nothing. Can be done with mutually exclusive locks, so that only one thread enters a critical section at a time.
- Condition Synchronization: Delay some operation until some precondition holds

15.6.3 Synchronization vs Parallelism

Synchronization is needed to ensure the correctness of the computation, and it reduces the amount of possible parallelism

15.6.4 Test-and-set

Instruction that sets boolean to true, and returns if it was false before. Used in spin-locks.

Avoids contention caused by repeated writes. Spin lock implementation below

```
boolean L = false

procedure acquire_lock(L)
  while not test_and_set(L):
    while L:
      // nothing (spin)

procedure release_lock(L)
  L = false
```

15.6.5 Synchronization Constructs in PLSs

Sync in Parallel Loops : Usually the responsability as is moved to the developer, but some PL's have implicit sync through **forall** in Fortran 95. For a specific line, first all the reads are done on the LHS, then the writes

15.6.6 Implicit Synchronization

The compiler determines dependency between concurrently executed code fragments. Then automatically add sync if it's needed. In practice, it is extremely hard and **Auto-parallelization** is mainly an open problem.

15.6.7 Memory Consistency

When multiple locations are written concurrently, when do the writes become visible to other threads?

Usually, we expect **sequential consistency**

- Each thread executes its instructions in it's own order
- Reads and writes are done inmediatly, shared memory acts like a global array

Relaxed Memory Models : In practice, reads and writes may happen out of order. This is because ensuring sequentiality is often inefficient. Compilers can and will reorder instructions.

Memory Models of different PLs : The issue is that different hardware implies different reordering behavior. PL's want to make it consistent within their own PL. So each PL defines it's own memory model

- Java Memory Model: By default, writes to shared objects are not inmediatly visible to other threads. You can force visibility with explicit synchronization, marking fields as volatile, or ordering writes and reads via a synchronized block.
- C11 Memory Model

They can add **fences**, instructions to sync memory access

15.7 Monitors

They are objects with operations, internal state and condition variables.

- One operation is active at any given time
- Calls to a busy monitor are delayed until the monitor is free.
- Operations can wait on a condition variable
- Operations can signal a condition variable, to allow others to resume

15.7.1 Monitors in Java

Every object can provide a monitor, **synchronized** keyword to acquire and remove locks.

15.8 Conditional Critical Regions

They are syntactically delimited, and represent a critical section. A condition must be true before entering the region. They are permitted to access a protected variable.

Syntax (pseudo code)

```
region protected_var when condition do

end region
```

15.9 Transactional Memory

They achieve atomicity without locks. What they do is

1. Speculatively execute the code
2. Check for conflicts (i.e race condition)
3. If conflict, commit the result, else roll back and attempt later.

Implemented in Java in the Multiverse Library

16 Functional languages

16.1 Functional Paradigm

It's an alternative to imperative PLs. The output is the matematical function of input, there is no internal state, no side effects.

In practice though, there are some fuzzy boundaries. There are functional features in many imperative PL's (higher order functions) and vice versa(assignment and iteration)

16.2 Historic origins

- Lambda Calculus
- Expressing computation based on abstraction into functions and function application[9]

16.3 Features

1. Functions assigned to variables, passed as arguments or return values.
2. Extensive Polymorphism (use function on different kind of values)
3. List types and operators
4. Structured function returns
5. Constructors for structured objects
6. Garbage collection

16.4 Purely Functional PLs

The functions depend only on their parameters, no other global/local state. The order of evaluation is irrelevant.

Example: Haskell

16.5 Non-Pure functional PLs

They have a mix of functional features with assignments, like **Scheme**, which is a dialect of Lisp, or OCaml

16.6 Function Application

Represented as a pair of parenthesis. First expression is the function, remaining expressions are arguments

(+3 4) represents 3+4 = 7

16.7 Creating Functions

Evaluating a lambda expression yields a function. First argument to lambda: Formal parameters. The remaining arguments are the body of the function

Example

$(lambda\,(x)\,(*\,x\,x))$ Function that squares value

16.8 Bindings

Names are bound to values through **let**. The first argument is a list of name,value pairs, and second argument are expressions to be evaluated in order

```
(let (  (a 3)
        (b 4)
        (square (lambda (x) (* x x)))
        (plus +))
     (sqrt (plus (square a) (square b))))
```

This yields 5.0

16.9 Conditional expression

Simple conditional expression with if

- First argument: Condition
- Second/third argument: Value returned if condition true/false

We can also do multiway conditional expression with **cond**

```
(if (< 2 3) 4 5)
```

And with cond

```
(cond
((< 3 2) 1)
((< 4 3) 2)
(else 3))
```

16.10 Dynamic Typing

Types are determined and checked at runtime

```
(if (> a 0) (+ 2 3) (+ 2 "foo"))
```

This evaluates to 5 if a is positive, else you get runtime type error

16.11 Lists

It's a **key** data structure in functional programming. It has various operations

- car extracts first element
- cdr extracts all elements but the first
- cons joins a head to the rest of a list

Examples

```
(car ' (2 3 4)) -> Yields 2
(cdr ' (2 3 4)) -> Yields (3 4)
(cons 2 ' (3 4)) -> Yields (2 3 4)
```

Note: The quote is to prevent interpretation from something like a literal

16.12 Assignments

We can achieve side effects via

- set! for assignment to variables
- set-car! for assignment to head of list
- set-cdr! for assigning tail of list

```
(let   ((x 2)
       (l ' (a b)))
       (set! x 3)
       (set-car! l ' (c d))
       (set-cdr! l ' (e))
       (cons x 1))

  Yields (3 (c d) e)
```

16.13 Sequencing

Cause interpreter to evaluate multiple expressions one after another with **begin**

```
(let
  ((n "there))
  (begin
    (display "hi ")
    (display n)))
```

16.14 Iteration

Looping through **do**

```
((lambda n)
(do ((i 0 (+ i 1)) -> For i = 0; i += 1
    (a 0 b) -> List of triples that each specify new variable, initial value,
     and expression to compute next value
    (b 1 (+ a b))) -> Another triple
  ((= i n) b) -> Termination condition with return value b
  (display b) -> Body of loop
  (display " "))) 5) -> Body of loop
```

16.15 Programs as Lists

It's the same syntax for programs and lists. They are called **S-expressions**. String of symbols with balanced parentheses

We can construct and manipulate an unevaluated program as list. Evaluation with **eval**

```
(eval (cons '+ list( '2 '3)))
```

16.16 Evaluation order

There are two main strategies

1. Applicative order: Evaluate arguments before passing them to function
2. Normal order: Pass arguments unevaluated and evaluate once used

Scheme uses Applicative order

Evaluation order also affects correctness. (Runtime error when evaluating an 'unused' subexpression).

16.16.1 Lazy Evaluation

Evaluate subexpressions on-demand, to avoid re-evaluating the same expression we memorize the result. In Haskell, that has no side effects, it's transparent to programmer. In PL's with side effects (Scheme), Programmer can explicitly ask for lazy evaluation.